Embedded System Implementation for Material Recognition Using Deep Learning

Khaled S. Younis, Waed Ayyad, Abdallah Al-Ajlony
Computer Engineering Department, School of Engineering,
The University of Jordan
Amman, Jordan
younis@ju.edu.jo, engwaedayyad@yahoo.com, alajlony@ieee.org

***Abstract*— Material recognition is the process of classifying materials into different categories based on the constituent material of the object under study. It is a very important problem in many fields and especially in the industrial field. In factories that use production lines to manufacture their products, one step is to separate materials and package different materials. In that case, it helps to move each material into specific containers. This step can be automated, which saves money, time and can improve efficiency. We utilized the advances in deep learning to build a system for material recognition, and we tested it on the Flicker Materials Database (FMD) with an accuracy of 79.25%. We also built a local database using materials from our environment via a camera mounted on an Raspberry Pi 3 embedded system, and that gave an accuracy of 90.5%. The system is self-contained and can be portable in any factory with minimal changes.**

Keywords— (Material Recognition; Convolutional Neural Networks; Deep Learning; Raspberry Pi 3; Computer Vision).

I. INTRODUCTION

Material recognition is an important problem related to many aspects in real life, so it is necessary to recognize materials in our surrounding environment. Automatic material recognition systems can be useful in many fields; one example is the industrial field. In factories, in many production lines there is a step to sort the products each in different containers. This process requires workers to do it by waiting until the product becomes ready then sorting it. This process consumes time and money.

Material recognition is the process of identifying materials around us into their categories, such as plastic, wood, fabric, etc. This step identifies the material from a single RGB image, by entering it into a trained convolutional neural network then analyzing it using deep learning concepts and finally classifying it into the appropriate category.

Deep learning (DL) and neural networks are methods used to analyze data. Neural network use software to mimic neurons in the human brain and have been shown to deliver state of the art results in numerous applications. DL is the implementation of neural networks with more than a single hidden layer of neurons. DL now gives the best solutions to many problems in image classification, speech recognition, and natural language processing [1].

There are several frameworks for deep learning. One of the most popular libraries is Keras [2] which is a high level API that is built on top of TensorFlow. Keras has a flexible architecture that allows distributed computations on one or more CPUs or GPUs in a desktop, server, or mobile device with a single API [14]. In this paper, we will present the results of utilizing Keras for building a Convolutional Neural Network (CNN) for solving the problem of material recognition.

In this paper, we propose an automated material recognition system, a very useful application to make this process easier, which gives better performance than the traditional methods of classifying materials and improves efficiency.

The rest of the paper is organized as follows: Section II presents a short survey of some related work. Section III summarizes the concepts employed in this paper. Section IV summarizes our methodology in utilizing DL and convolutional neural networks (CNN) in the material recognition problem. Section V presents the results and discussions of these results. Finally, section VI presents conclusions and discusses future work.

II. RELATED WORK

This section will summarize the most significant published works on the material recognition problem and embedded system implementations.

Many studies rely on material recognition to be part of visual recognition as in the work of Hu et. al. [3]. They used a set of local features of kernel descriptors and evaluated both standard features (e. g. shape and color) as well as a new set of features that are related to materials (e. g. variances of gradient orientation and magnitude) to capture as much material properties as possible. The accuracy was 54%.

Badami and Reinhard [4] used a pool of features that cover several appearances. For a fixed camera and object position, the image can be identified using many features, such as surface structure, color, object shape and environment illumination. After the features were calculated, they built the material recognition system by first quantizing the features then using a support vector machine (SVM) for material recognition. SVMs have become popular for classification tasks because they achieved many advantages, such as the ease of generalization of the problem, the ability to handle high-dimensional feature space and the absence of the local minima. The Flickr dataset and the KTH-TIPS dataset are used to evaluate the recognition task. The KTH-TIPS [5] dataset consists of 11 different materials: Crumpled aluminum foil, Cork, Wool, Lettuce leaf,

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©2017 IEEE

978-1-5090-5969-0/17/$31.00 ©2017 IEEE
Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 26,2026 at 09:06:20 UTC from IEEE Xplore. Restrictions apply.

Corduroy, Linen, Cotton, Brown bread, White bread, Wood and Cracker. The recognition rate achieved was 99.4% on KTH-TIPS while 53.6% was achieved on the Flickr dataset. This demonstrates that the Flicker Dataset is very difficult to classify.

In [3], the authors evaluated their approach on three datasets; 1) Flickr dataset, 2) A set of seven thousands images collected from Image Net - including 7 materials and 76 object categories- and 3) Macro dataset. They evaluated all five kernel descriptors on the three datasets. For the Flickr dataset, they took average accuracies over 5 trials. For the Image Net and Low-res Macro datasets, they reported accuracies over 5 trials as well as randomly partitioned training and testing data.

Using the Flickr dataset, they achieved 54% by combining 5 descriptors. In the ImageNet dataset, they used two sub groups of data material-1 and material-2; material-1 used all objects in both training and testing; material-2 used half of the object categories (5 per material) in training and half in testing. Accuracy in material-1 was higher than material-2 since material-2 lacks object information. Using the 5 descriptors provides much larger improvement compared to Flickr since the dataset is larger.

In this work [6], a new dataset was introduced called MINC (Materials in Context Database). Bell and et al utilized MINC with deep learning concepts to achieve material recognition and segmentation of images. So, different CNN architectures were applied to achieve material recognition and segmentation in full image. AlexNet, GoogleNet and VGG-16 are used. Also, 100 random images were taken from MINC to be the same as FMD dataset categories; in Alex Net, they achieved 64.6. In FMD experiment using their work on AlexNet, they achieved 66.5%.

So far, we have discussed the work done in material recognition from a software, algorithmic point of view. In the following paragraphs, we will discuss what previous work that was done on the hardware implementation of material recognition and sorting using embedded systems.

Sorting objects is usually done manually using human labor. Recognizing a particular object and placing it in the required position is tiring work, especially in the industrial field where the sorted objects are very heavy. In the work of [7], the authors are considering all these factors along with the cost to make the process more efficient. They used Open-Source environments to make it cost effective. Linux based Operating system used in Raspberry Pi (Raspbian OS), GNU Octave, Python and Arduino IDE are freely available.

The purpose of their system was to provide a robotic arm that sorts the objects in the most efficient way and also in a cost effective manner. They sort objects of three different shapes and colors, i.e. Square, Circle and Triangle (as seen from the Top-View of a Cube, Sphere/Cylinder and Triangular-Prism aligned vertically) and RGB colors i.e. Red, Blue and Green respectively [7].

In [8], the authors mentioned that existing systems distinguish between different objects mainly using color. This technology can be used to handle different materials in the logistics and packaging industry where the objects moving through a conveyer belt can be separated using a color detecting robot. The detection of the particular color is done by light intensity to frequency converter method, which consists of a robotic arm that is supervised by a microcontroller based system. It aims to classify the objects coming on the conveyor belt by color and size, then picking up and placing the objects in the desired place, thereby eradicating the repetitious work done by humans, achieving accuracy and speed in the process.

Color sorting systems in segregating crayons use visual color matching or color sorting techniques. For example, the crayon manufacturer guarantees that the green crayon in each color-assorted box will look identical. In mineral sorting, the system scans bulk material in free fall by means of a color line camera. The respective data is evaluated by free programmable sorting software of an industrial computer by color, brightness and size into good products and rejected parts [8].

Raspberry Pi (RPI) is a low cost, portable, multipurpose and tiny computer. The latest version of it is Raspberry Pi 3, which is used in this system. Figure 3 shows Raspberry Pi 3 interfaced with a web camera, dc motor and linear actuator. A 5V and 2A power supply is given to the Raspberry Pi to work. Raspberry Pi 3 has 4 USB ports which can be used for connecting a webcam, and it has 40 GPIO pins which are connected to the DC motor and linear actuator. We use a H bridge to drive the DC motor and linear actuator. Pulse width Modulation (PWM) is used for controlling the speed of the DC motor. H bridge is used as it is available at a low cost and can be used to run the motor in one direction or the other. Since H bridges generally support 2 motors, we can use the other part of the Hbridge for driving the linear actuator [8].

III. THEORY

This section summarizes the main concepts that were implemented in this paper.

A. Deep Learning

Deep learning is a branch of machine learning, employing numerous similar, yet distinct, deep neural network architectures to solve different problems in natural language processing, computer vision, and image classification.

B. Convolutional Neural Network

CNNs are a type of neural network that are applied in many fields and have proved to provide very efficient solutions to many problems, such as identifying faces and objects and OCR. CNN takes its name according to the convolution operation done through its layers . Convolution is multiplication operation of each pixel in the image with each value in kernel, which is another matrix. The key advantage of using the convolution operation is generating many images from the original image thereby enhancing the features extracted from the original image, which leads to making the classification process more powerful. CNN is a sequence of layers, and each layer has different task [9].

The CNN layers are summarized below:

1) Convolutional and non-linearity: The objective of this layer is to extract features from the input image. Initially, the CNN does not know where exactly the features will match in the image, so it tries to find them everywhere in the image by

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 26,2026 at 09:06:20 UTC from IEEE Xplore. Restrictions apply.

using a matrix called filter. The filter represents a specific feature. CNN applies convolution operations using a sliding filter in the image and multiplies each pixel in the image with each value in the filter. Then, this operation is repeated for other features (filters), and the output of this layer will be a set of filtered images. An additional operation, called ReLU, has been used after every convolution operation. ReLU stands for Rectified Linear Unit and is a non-linear operation. ReLU is an element wise operation (applied per pixel) to introduce non-linearity in our network. Since convolution is a linear operation – element wise matrix multiplication and addition, we add nonlinearity using ReLU; this operation converts each negative pixel in the feature map to zero. ReLU functions better in most situations [9].

2) Pooling or subsampling layer: The pooling layer is also called subsampling or downsampling; it reduces the dimensionality of each filtered image but preserves the most important information. Pooling can be of different types: Max, Average, Sum, etc. The output will have the same number of images, but they will each have fewer pixels. This is also helpful in managing the computational load [9].

3) Fully connected layer: In CNN architecture, after alternating between convolutional and pooling for a few layers, the convolutional features are reshaped to a vector that will be used in fully connected layer. Fully connected layers are the basic building blocks of traditional neural networks. They treat the input as one vector instead of two dimensional arrays. Fully connected implies that every neuron in the previous layer is connected to every neuron in the next layer. The output from convolutional and pooling layers represents high level features, and fully connected layers use them to classify materials (input images) into the appropriate class based on the training of the dataset [9].

IV. METHODOLOGY

This section summarizes the methodology used in this study for recognizing materials and characterizes the implementation of our system. The methodology involves three subsections: Sources of data, System design workflow and training process, and Embedded system implementation.

A. Sources of Data

In our work, we used two datasets. The first one is the FMD dataset [10]; FMD stands for Flickr Material Dataset. It consists of ten common material categories: fabric, foliage, glass, leather, metal, paper, plastic, stone, water and wood. There are 100 images and masks in each category, 50 close-ups and 50 regular views as shown in Figure 2.

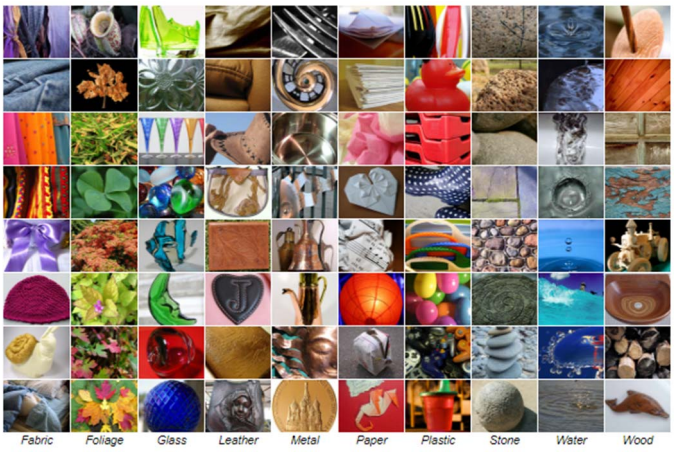


Fig. 1. FMD Dataset

The second one is the (UJIS) dataset; UJIS stands for University of Jordan Inspection System. UJIS was used to apply our model to a real database from our environment in order to make the classification application more real. This data set consists of the same ten classes as the FMD dataset. Some examples illustrated in Figure 2 below:

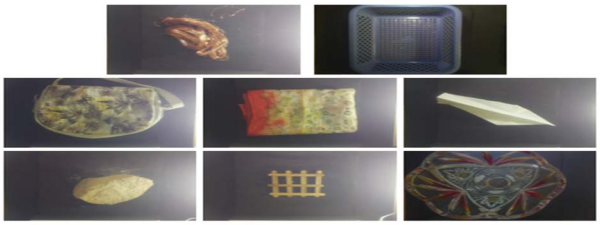


Fig.2. UJIS dataset sample

B. System Design Workflow Figure *4* illustrates our system design workflow. The workflow begins by accessing the database and performing the required preprocessing then feature extraction done by the convolutional neural network. These features were used to train the DNN to get proper results for the category of the material that was placed in the inspection room.

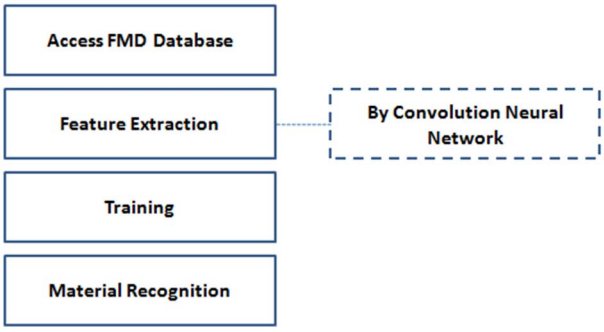


Fig. 3. System Design Workflow

1) Access FMD database and preprocessing: In our work, we did preprocessing in the FMD dataset [10]. Two experiments were done. In the first one, the mask was used as in [1] to exclude the background from images. The mask is a binary image with the same size of the actual image but its pixel values are only zero and one (only black and white) The mask

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 26,2026 at 09:06:20 UTC from IEEE Xplore. Restrictions apply.

was used to separate material from background by applying bitwise AND operation between the image and its mask. In the second experiment, the mask wasn't used, only images were used then the following steps were continued in both experiments.

2) Feature extraction: This step was done by entering input (images) details (Rows, Column) to the network, which was used to do features extraction. We used VGG 16; this network was pre-trained on the ImageNet dataset, which is a huge dataset used in image classification tasks. VGG 16 was used to benefit from low level features that were extracted from the ImageNet dataset because in most image classification tasks these features were not changed. It was done by freezing lower layers, and only higher layers were updated according to our problem, by adding a fully connected layer with 10 neurons (10 classes). The VGG 16 network is shown in figure 4.

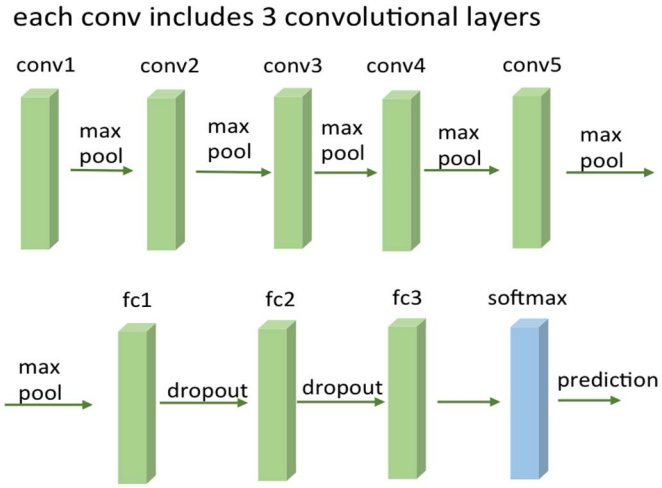


Fig. 4. VGG 16 architecture

Deep neural network was used by adding fully connected layer, dropout layer which is a technique used in neural networks where randomly selected neurons are ignored during training; they are dropped out randomly. This means that dropout neurons will not contribute to training. This technique is usually used to make a network more generalized and to not causing over fitting. Overfitting is a common problem in deep learning and neural networks. It happens when the model gets good result in training data, but it gets bad result in new data since the model not generalized.

3) Training process: Data was divided into training, testing and validation. Training was started by first compiling the model and determining some parameters like optimization function, loss function and metrics. Then training data was fitted in the model and determined other parameters like epoch and batch size. Then training was started. Training was continued and epochs were changed until the best results were obtained. The process of training FMD dataset is illustrated in figure 5 below:

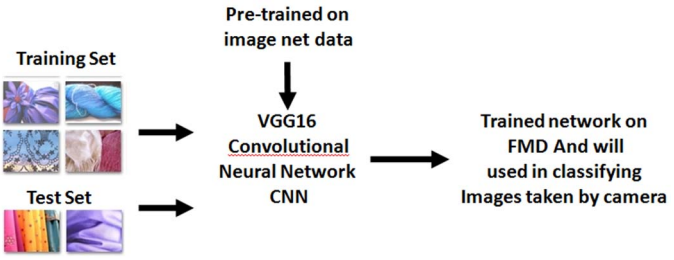


Fig. 5. Training Process

4) Material recognition: After VGG 16 was trained on the training dataset, it was used to classify testing data and compute its accuracy to evaluate the performance. We noticed that using a mask was not a required step in our approach compared with the results in [4] because similar results were achieved in both experiments.

C. Embedded System Implementation

A microcontroller is a small entire computer implemented on an integrated circuit that incorporates all the features that are found in microprocessor. For instance, microcontrollers are used in different applications as controllers in automobiles and as system exposure, smart houses, office machines, power tools and other embedded systems. microcontrollers are powerful digital processors, the degree of control and programmability they provide significantly enhances the effectiveness of the applications [12].

The first and main component used in the project is the Raspberry Pi 3. It's one type of the microcontrollers used to control the system, operate it on python programming language with the used libraries Keras and Tensor Flow [13].

There are two roles for the Raspberry Pi 3 Camera Module. The first one is used to prepare the UJIS Database by taking photos of every class of material and storing it. The second one is used to take a photo of the object during operation and testing. We used this camera because of its high quality pictures. It supports 1080p30, 720p60 and VGA90 video modes, as well as still capture [15]. To create effects and to control the camera module, there are numerous third-party libraries built for it, including the Picamera Python library [15].

In building the inspection room, we used a wooden box with 40x30x40 cm dimensions. This room contains the hardware components: The Raspberry Pi 3, Arduino, power bank, camera module, wires and LEDs. Figure 7 illustrates the inspection room.



Fig. 6. Inspection Room with a glass object inside

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 26,2026 at 09:06:20 UTC from IEEE Xplore. Restrictions apply.

The inspection room was built to hold one object to capture an image of it using the camera module. This room acts as a part of an integrated system in a factory before the classification operation. The object passing through this room takes required operations to make decisions in the next step. Figure 7 illustrates the inspection room hardware components.

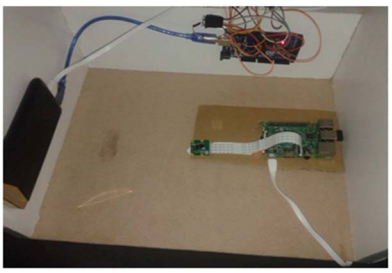


Fig. 7. Inspection Room Hardware

The goal of our work is to classify different materials; hence, the stages of processing are as follows. First, we put the object we want to classify in the inspection room, then the camera that is connected to the Raspberry Pi 3 takes a photo and stores it. The code, which used the Tensor Flow library for deep learning operation, will process the image and decide which class of the UJIS the object belongs. Finally, when the result becomes ready, the Raspberry Pi sends a signal to Arduino to show the output on the LEDs. This is basically the same signal that can be used to control a real industrial arm in a factory to do the required manipulation of an object and put it in the suitable container.

V. RESULTS AND DISCUSSION

This section presents and discusses the results obtained from our experiments on the FMD database and the UJIS database. It also compares our results with other results achieved in this research area.

A. Results of Deep Learning on FMD

We started training the VGG 16 network after freezing first 25 layers of the based network trained on ImageNet dataset and fine-tuning the rest layers in addition to the added 256 fully connected layer and the 10 neurons output layer using a small number of epochs and then increased it until the best performance was achieved, which is 79.25% when the number of epochs are 150, but our training process was stopped before 150 since the early stopping technique was used, which stops training according to some conditions we specified. In our case, we defined it to stop the process if the validation loss did not decrease for 10 consecutive times.

Table I shows the confusion matrix of our experiment on the FMD database. The confusion matrix was used to evaluate accuracy by adding the diagonal values and then dividing them by the total number of test data. The diagonal values represent the correct prediction for each category while the non-diagonal represent the misclassification for each category.

TABLE I. CONFUSION MATRIX ON FMD DATASET

	Fabric	Foliage	Glass	Leather	Metal	Paper	Plastic	Stone	Water	Wood
Fabric	27	0	1	2	2	3	1	0	2	1
Foliage	2	37	0	0	0	1	0	0	0	1
Glass	0	0	27	0	3	0	1	0	3	0
Leather	1	0	0	27	1	1	0	0	4	1
Metal	2	0	5	5	22	0	0	2	2	1
Paper	3	3	1	0	3	37	2	0	2	1
Plastic	3	1	2	4	1	0	25	0	0	0
Stone	0	0	3	1	0	0	1	27	0	1
Water	0	0	0	0	0	0	0	0	43	1
Wood	1	0	0	2	3	1	0	4	0	41

B. Results of Deep Learning on UJIS Dataset

We applied the same procedure on the UJIS database. The only difference was that water and foliage classes were excluded from the database since these materials are not as important in the industrial field as other classes, hence, the output layer made up of 8 neurons instead of 10. The best result achieved was 90.5%, which was considered state of the art performance because this data was prepared from our surrounding environment. The confusion matrix of UJIS is shown in Table II below.

TABLE II. CONFUSION MATRIX ON UJIS DATASET

	Fabric	Glass	Leather	Metal	Paper	Plastic	Stone	Wood
Fabric	12	0	0	0	0	0	0	0
Glass	0	18	0	0	0	2	0	0
Leather	0	0	24	1	1	0	0	0
Metal	1	2	1	28	1	0	0	0
Paper	2	0	1	1	32	0	1	0
Plastic	0	1	0	2	0	22	0	0
Stone	1	0	0	1	0	0	40	0
Wood	0	0	0	2	0	0	0	25

C. Loss function

The loss function was used to calculate the difference between the actual class and the predicted class; in our training, we try to decrease it. Figure 8 shows the relation between the number of epochs and the loss function on the FMD dataset.

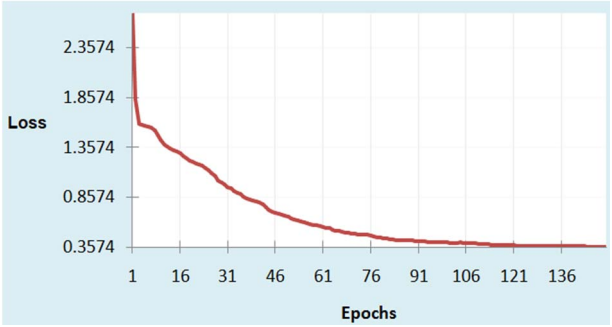


Fig. 8. Loss Function

D. Performance Summary

Table III summarizes our work on FMD and the other works in the research area.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 26,2026 at 09:06:20 UTC from IEEE Xplore. Restrictions apply.

TABLE III. PERFORMANCE SUMMARY

Author	Method	Classification Accuracy
Badami and Reinhard [4]	SVM	53.6%
Hu et. al. [3].	LMNN on kernel descriptors	54%
Bell et. al [6]	CNN (AlexNet, GoogleNet and VGG-16)	66.5%
Proposed work	CNN DL	79.25%

E. Validation Accuracy

As the number of epochs increase, the validation accuracy increases as shown in Figure 9 below:

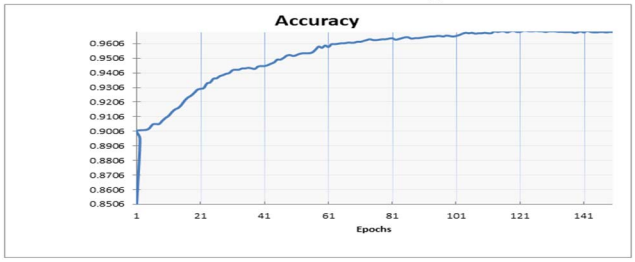


Fig. 9. Validation Accuracy

F. Effect of changing test ratio

Table IV presents the effect of changing the ratio of test data from whole data. The results below were calculated using the confusion matrix for each case as we calculated test accuracy in section A. Accuracy decreases as the test ratio increases; as shown in table 4, the best we achieved was 79.25% when the test ratio was 20%.

TABLE IV. THE EFFECT OF CHANGING TEST RATIO

Percentage of test data	20%	40%	50%
Number of test data	400	800	1000
Accuracy	79.25%	74.50%	72.30%

VI. CONCLUSION

In this paper, a new material recognition system is introduced that is different from other approaches in the literature. It was proven that the proposed system is highly efficient in term of accuracy and its ability to recognize materials without the need to even use the mask in the FMD database.

We applied deep learning and convolutional neural network with data augmentation to double the size of the database. We used the transfer learning concept by using a pre-trained network on a large dataset called Image Net and fine-tuned the model using our dataset. VGG 16 architecture was fine-tuned by removing the higher layer and replacing it according to our problem. We studied the effect of changing the ratio of test data on the accuracy. We first used 20% of the data as test data and then increased it to 40% and finally to 50%; the best accuracy is 79.25% which we achieved using 20% ratio as test data.

Finally, we went the extra step of implementing the DL on an embedded system, Raspberry Pi 3 with a camera for controlling the size of the device. Furthermore, we tested the performance on realistic scenarios using our own database. The database is considered large and uses actual, real-life materials.

REFERENCES

[1] M. Nielsen. (2017, June 21). Neural Networks and Deep Learning. Retrieved from http://neuralnetworksanddeeplearning.com/index.html

[2] (2017, June 21). Keras: The Python Deep Learning Library. Retrieved from https://keras.io

[3] D. Hu, Liefeng Bo and Xiaofeng Ren. Toward Robust Material Recognition for Everyday Objects. In Jesse Hoey, Stephen McKenna and Emanuele Trucco, Proceedings of the British Machine Vision Conference, pp. 48.1-48.11.

[4] I. Badami, and R. Klein. "Material recognition: Bayesian inference or SVMs?." Seminar on Computer Graphics (CESCG). 2012.

[5] B. Caputo, E. Hayman, and P. Mallikarjuna. "Class-specific material categorisation." Computer Vision, 2005. ICCV 2005. Tenth IEEE International Conference on. Vol. 2. IEEE, 2005.

[6] S. Bell., P. Upchurch., N. Snavely., & K. Bala. (2015). Material recognition in the wild with the materials in context database. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 3479-3487).

[7] V. Pereira, V. A. Fernandes and J. Sequeira, "Low cost object sorting robotic arm using Raspberry Pi", 2014 IEEE Global Humanitarian Technology Conference - South Asia Satellite (GHTC-SAS), Trivandrum, 2014, pp. 1-6.

[8] S. Sheela, K. R. Shivaram, S. Meghashree, L. Monica., A. Prathima and S. .M. Kumar, "Low Cost Automation for Sorting of Objects on Conveyor Belt", International Journal of Innovative Research in Science, Engineering and Technology. 10, May 2016.

[9] (2017, June 21). A Beginner's Guide To Understanding Convolutional Neural Networks Retrieved from https://adeshpande3.github.io/adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks/

[10] L. Sharan., R. Rosenholtz., and E. H. Adelson. (2017, June 2). Flickr Material Database (FMD) Retrieved from https://people.csail.mit.edu/celiu/CVPR2010/FMD/

[11] Liu, Ce et al. "Exploring Features in a Bayesian Framework for Material Recognition." IEEE, 2010 Computer Vision and Pattern Recognition (CVPR), IEEE Conference on, pp. 239–246.

[12] L. Ajao & O. M. Olaniyi & J. Kolo & A. Ajao. (2015). Project-Based Microcontroller System Laboratory Using BK300 Development Board With PIC16F887 Chip. International Journal of Embedded Systems and Applications. 5. 15-28. 10.5121/ijesa.2015.5302.

[13] (2017, June 21). Raspberry Pi 3 Model B - Raspberry Pi. Retrieved from https://www.raspberrypi.org/products/raspberry-pi-3-model-b/

[14] Abadi, Martín, et al. "Tensor Flow: Large-scale machine learning on heterogeneous distributed systems." arXiv preprint arXiv:1603.04467 (2016).

[15] (2017, June 21). Camera module v2. Retrieved from https://www.raspberrypi.org/products/camera-module-v2

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 26,2026 at 09:06:20 UTC from IEEE Xplore. Restrictions apply.